Task 1:

Suppose we have a list of words, such as `["Apple", "Banana", "Plum", "Apple", "Banana", "Apple"]`. Write a function, `word_counts(word_list)` that counts how many times each word appears, and returns a dictionary of those counts. For example, for the list above, the function should return `{"Apple": 3, "Banana": 2, "Plum": 1}`.

The signature of this function is `(list) ---> dict`.

Task 2:

Suppose we're trying to take stock of where we're spending our money, and we want to use the computer to help us out. We'll write a function, `greatest_expense(items_purchased)` to help us with this. This function takes a dictionary, as follows:

- Keys in the dictionary are all of the things we purchased that month; for instance, `"bananas", "eggs", "frozen pizzas"`
- Values in the dictionary are tuples, bundling together the cost of each item and the number of that item we purchased; for instance, `(.17, 25), (.12, 100), (4.5, 4)`.

For our example above, we spent $4.25 (25*.17) on bananas, $12 (100*.12) on eggs, and $18 (4*4.5) on frozen pizzas. This makes frozen pizzas our largest expense, at 52.55% of the budget.

Your function should print the name of the item that was our single greatest expense, and the percentage of our total expenses from that item. The format expected is:

```
The greatest expense came from ITEM, which accounted for PERCENT of the total amount spent.
```

`ITEM` is, of course, the name of the item. `PERCENT` is its percentage contribution, and should be formatted to two decimal places.

The signature of this function is `(dict) ---> None`

Task 3:

In this task, we'll write a function, `invert_dictionary(some_dict)`. By invert, this means to swap the (key, value) pairs, so that `{"a": 30}` becomes `{30: "a"}`. However, there's a catch here.

Recall that keys in a dictionary must be unique, but values do not. Thus, to make sure we handle collisions elegantly, each mapping in the dictionary will associate a key (the old value) with a *list* of all of the values (the old keys) that were present.

You can assume that all of the values in the original dictionary are legal keys for the new one.

Example:

`invert_dict({"a":10, "b": 10, "c": 25})` should return `{10: ["a", "b"], 25: ["c"]}`.

The signature of this function is `(dict) ---> dict`.